



AGENT.md: 给 AI 员工的入职合同

—— 数字生命卡兹克 GPT-6 Astra 指令文件模板拆解 + 真机 A/B 对比实验 (2026-10-03)

来源: x.com/Khazix0918

166K 浏览 · 2.9K 收藏

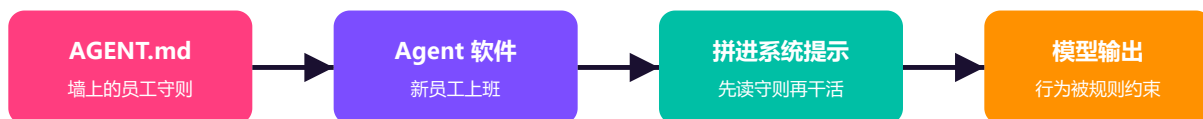
实验: glm-4.6 真实调用 18 次

一份 **贴在仓库/配置里、AI 每次开工都先读一遍的“行为契约”**。本指南用 12 个疑问拆解它，并用真机实验证明：**模板里的规则不是玄学，贴上去模型行为真的 measurable 地变了**。

📌 疑问清单 (本篇目录)

Q1 AGENT.md 到底是个啥？谁在什么时候读它？

它就是一个纯 Markdown 文本文件，里面写着“你干活时要遵守哪些规矩”。**Agent 软件每次接到任务时，会自动把它塞进系统提示 (system prompt) 里**——相当于员工每天上班先看一眼贴在墙上的《员工守则》再开始干活。



关键认知：模型本身没有“记性”，每次会话都是新员工——所以规矩必须**每次都重新发一遍**，AGENT.md 就是那个被反复发的固定文本。

Q2 为什么写成文件，而不直接把规则写死在代码里？

三个理由：① **用户可改**——不用改软件源码就能定制自己的 agent；② **可版本管理**——git diff 一眼看清这条规矩哪天改的；③ **可移植**——换 agent 软件 (Astra / Claude Code / Codex) 文件跟着走。这和知识库里的 [[DESIGN.md 规范文档]] 是同一招：LLM 的母语是 Markdown，纯文本规则零解析成本、人机共读。

Q3 模板为什么是这 6 个板块？各管什么？

板块	管什么	一句话人话版
沟通	输出风格	说中文、先结论、少列表、不废话
指令优先级	听谁的	安全 > 当前指令 > 项目规则 > 全局规则 > 记忆
执行方式	干活姿态	开工别磨叽、先干完再问、别迎合、别瞎警告
测试与验证	验证力度	改动越大测越多，改个错别字别跑全套 CI
工具与并行	干活手法	用 rg 搜、批量读、子代理只在真独立时才派
规则来源	唯一事实源	规矩只放一份 AGENTS.md，别的文件只引用不复制

这 6 块正好覆盖一个 AI 员工的完整生命周期：怎么说话 → 听谁指挥 → 怎么干活 → 怎么自证 → 用什么工具 → 规矩本身放哪。缺哪块，哪块就退回模型默认行为（往往是唠叨、过度谨慎、过度测试）。

Q4 "指令优先级"为什么排第二？听谁的到底有多重要？

Agent 工作时上下文里飘着好几种指令：系统提示、你的全局偏好、项目里的 AGENTS.md、历史记录、Skill 文档……**不排优先级，冲突时模型就只能“猜”，猜错一次就是一次事故**。模板给出的序：



这就是操作系统的**中断优先级**思想搬进提示词工程：越靠近“当下、具体”的指令优先级越高，越“陈旧、泛化”的越靠后。与知识库 [[提示词权威边界]] 互补——那边管“一段提示词内部谁有权改哪部分”，这边管“多层指令之间谁听谁”。

Q5 "先干完再问""别迎合""别瞎警告"在治什么病？

这三条全是真实使用中的高频痛点：① **磨叽病**——干两步就停下来问“可以继续吗”，人得全程陪聊；模板规定“已授权的先完整做完，再统一提问”。② **迎合病**——用户说方案 A 其实不对，模型顺着夸；模板规定“不适合就直接说，不为迎合改口”。③ **警告泛滥病**——随手写个玩具脚本，它给你上一堂 SQL 注入安全课；模板规定“不为假想风险加未经请求的警告”。（这条我们实验里实测到了，见下方场景 3）

Q6 测试还要"按改动大小分级"? 全测不是更安全吗?

全测当然"安全", 但 agent 是高频干活机器: 每改一行注释都跑 10 分钟 CI, **时间全烧在仪式上**。模板的分级: 琐碎可逆改动(注释/文案/格式)不强制测试; 测试力度与影响面成正比; 收尾清理临时文件。这与知识库 [[质检Gate与自我纠错循环]] 不冲突——Gate 是"该检的地方必须检", 分级是"**不该检的地方别烧钱**", 一个是质量下限, 一个是成本上限。

Q7 "工具与并行"板块: rg、批量读、子代理克制是在省什么?

① 搜索用 **rg**: 比递归 grep 快一个量级, agent 一天搜几百次, 累计省的是真实的挂钟时间。② 无依赖的读取**批量并行发起**: 串行 5 次网络往返 vs 并行 1 次。③ **子代理只用于真正独立的工作流**——乱开子代理 = 每个都要重新加载上下文, 又贵又慢, 最后还得主代理重新核实。共享状态、连续决策的事, 主代理亲自干。(呼应知识库 [[GitWorktree并行隔离]]: 并行的前提是物理隔离, 不隔离就别并行。)

Q8 "规则来源": 为什么 CLAUDE.md 只当"兼容入口"?

不同 AI 工具读的配置文件名不一样 (CLAUDE.md / AGENTS.md / .cursorrules...)。如果每个文件各写一份规矩, **改一条规则要改 5 处, 很快就会互相矛盾**。模板的解法是教科书级的**单一事实源 (SSOT)**: 真规矩只写在一份 canonical AGENTS.md 里, 其他文件只写一行"请见 AGENTS.md"。和知识库 [[源码即Prompt]] 的"发布期渲染"是同一思想的两个流派: 一个多文件复制渲染, 一个单源引用指向。

Q9 它和知识库已学的概念是什么关系?

已有概念	管什么	与 AGENT.md 的分工
[[角色提示]]	人格设定 (你是谁)	AGENT.md 管行为纪律 (你怎么干活), 不管人设
[[AgentSkills技能包]]	方法论包, 按需加载	AGENT.md 常驻上下文 管全局基线; Skill 干活时才翻开
[[DESIGN.md规范文档]]	项目 UI 视觉规范	同是"根目录 Markdown 契约", 一个管施工方一个管装修风格
[[提示词权威边界]]	Skill 内部谁改哪段	AGENT.md 的指令优先级管跨层指令谁听谁
[[提示词模板]]	带变量槽位的单条提示词	AGENT.md 无变量, 是固定的行为常量

Q10 规则真的有效吗？——真机 A/B 对比实验

方法：同一个 glm-4.6 模型，A 组裸系统提示，B 组加模板的"沟通+执行方式+测试与验证"三节；3 个场景 × 各 3 次；用**确定性正则打分**（不靠人眼）。完整脚本与原始数据：

`exercise\agentmd_ab_test.py`、`exercise\ab_result.json`

场景（测哪条规则）	指标	A 裸提示	B 带模板	结论
合并 txt（沟通·列表克制）	平均列表行数	3.33	0.00	列表克制见效
	结论先行率	0%	33%	弱正向
改错别字（测试分级）	结论先行率	0%	100%	"琐碎改动别搞仪式"见效
	首行风格	"取决于你的情况..." + 分支	"基本没有 了"	
玩具拼 SQL（不瞎警告）	未经请求警告	首行就是"⚠️ SQL注入警告"	0 次	效果最显著
	平均建议句数	2.33	0.00	

✅ **实验结论：**模板规则**不是玄学**——"少用列表""先给结论""别加未经请求的警告""琐碎改动不搞流程"四条在真机上全部可测量地改变了模型行为。另一面：常规单轮"写个函数"任务上两组差异很小，**因为这些规则治的病主要发生在长会话、多步骤的 agentic workflow 里**（磨叽、中途反复问、过度测试、临时文件残留）——单轮问答本来就没有那些病。

Q11 ⚠️ 双刃剑：压掉警告是福是祸？

实验里真实看到的另一面：B 组连 SQL 注入警告都不发了。对你自己的玩具库这是"清爽"，对真实生产库这是"灾难"。**安全红线应该放在系统层/平台层（模板第一条也是这么写的："遵守系统、平台和安全约束"），而不能指望"少警告"的用户指令兜底。**自己抄模板时，务必保留指令优先级板块，把安全约束钉在最顶层。

Q12 我想自己写一份，从哪几条开始？换 agent 要改什么？

起步三件套（性价比最高）：① 语言与结论先行（沟通 2 条）→ ② 指令优先级全节 → ③ "不为假想风险加未经请求警告"。写规则的红线：**每条规则必须对应一个你真实被烦到过的失败模式**，写"保持专业、高质量回答"这种装饰性规则等于没写（模型本来就想做到，做到了你看不出，做不到它也不当真）。**换 agent 要改的**：工具与并行板块是 harness 专属（rg / Chrome / lark-cli 换成你那家的等价物）；规则来源板块换成你那家的配置文件名。

🔧 动手实验：模板效果对比（已真机跑通）

步骤 1 · 准备材料：把推文里的模板全文存为 `exercise\AGENT_md_template.md`（逐字原文，别凭记忆转述——本实验第一版就转述错了，重抓原文才修正）。

步骤 2 · 配置 API：复用 `ANTHROPIC_AUTH_TOKEN` + `ANTHROPIC_BASE_URL` 代理接入 glm-4.6。坑：glm 返回 thinking block，要过滤且 `max_tokens ≥ 512`，否则全是思考没有正文。

步骤 3 · 跑对比：

```
python F:\reminder\agentmd_template\exercise\agentmd_ab_test.py
```

3 场景 × 2 组 × 3 次 = 18 次真实调用，正则确定性打分。

步骤 4 · 看结果：

```
type F:\reminder\agentmd_template\exercise\ab_result.json
```

✅ 预期：场景 1 列表行 A≈3.3 → B=0；场景 2 B 组结论先行 100%；场景 3 A 组首行"⚠️ SQL 注入警告"、B 组警告词 0。（实测全部命中，见 Q10 表格）

步骤 5 · 延伸玩：把 SCENARIOS 换成你自己被 AI 烦到过的任务（如"每次都让我写 CHANGELOG"），给模板加一条对应规则，重跑看它是否压得下去。

📋 这份模板给我什么（能力清单）

① 直接抄用

模板原文存于

`exercise\AGENT_md_template.md`，改掉 harness 专属板块即可贴进自己的 agent 配置。

② 判别好规则的标准

"规则 = 真实失败模式的解药"。拿这条标准可审计任何提示词/规则文件里的废话。

③ 一套提示词 A/B 实验法

确定性正则打分 + 多场景多 trial，成本 18 次调用，任何"这个 prompt 有没有用"之争都能实测裁决。

④ 指令分层思维

安全>当前指令>项目>全局>记忆——任何多源指令系统（不只 AI）都适用的仲裁序。

F:\reminder\agentmd_template · 拾遗任务 5bed5c97 · 2026-10-03 · 概念已入库 vault (Agent指令文件AgentMd / 指令优先级层级)